

Gestione eccezioni avanzata

Gestione eccezioni avanzata

Tecniche avanzate per gestire le eccezioni in Python.

Ripasso rapido

Hai già visto come usare `try/except` per gestire gli errori. Ora vediamo tecniche più avanzate per situazioni più complesse.

```
try:
    # codice che potrebbe dare errore
except TipoErrore:
    # cosa fare se c'è quell'errore
else:
    # eseguito solo se NON ci sono stati errori
finally:
    # eseguito SEMPRE, con o senza errori
```

Creare i tuoi errori personalizzati

Python ti permette di creare **errori su misura** per il tuo programma. È utile quando vuoi segnalare situazioni specifiche con messaggi chiari.

Pensa a un modulo di iscrizione scolastica: ha senso creare un errore `EtaNonValidaError` invece di usare il generico `ValueError`.

```

# Creo un errore personalizzato per età non valide
class EtaNonValidaError(Exception):
    """Segnala che l'età inserita non è valida."""
    pass

# Creo un errore personalizzato per voti non validi, con informazioni extra
class VotoNonValidoError(Exception):
    """Segnala che il voto inserito non è nell'intervallo 0-10."""
    def __init__(self, voto, messaggio="Il voto deve essere tra 0 e 10"):
        self.voto = voto          # Salvo il voto errato per poterlo
        mostrare
        self.messaggio = messaggio
        super().__init__(self.messaggio) # Passa il messaggio alla classe
        madre

# Funzioni che usano gli errori personalizzati
def controlla_eta(eta):
    if eta < 0 or eta > 150:
        raise EtaNonValidaError(f"L'età {eta} non è valida.")

def controlla_voto(voto):
    if voto < 0 or voto > 10:
        raise VotoNonValidoError(voto)

# Utilizzo
try:
    controlla_eta(-5)
except EtaNonValidaError as e:
    print(f"Errore: {e}")
# → Errore: L'età -5 non è valida.

try:
    controlla_voto(15)
except VotoNonValidoError as e:
    print(f"Errore: {e.messaggio} (voto ricevuto: {e.voto})")
# → Errore: Il voto deve essere tra 0 e 10 (voto ricevuto: 15)

```

Lanciare un errore manualmente con **raise**

Puoi **provocare** tu stesso un errore quando ti accorgi che qualcosa non va, anche senza che Python lo faccia automaticamente.

```
def dividi(a, b):
    if b == 0:
        raise ValueError("Il divisore non può essere zero.") # Lancio
l'errore
    return a / b

try:
    risultato = dividi(10, 0)
except ValueError as e:
    print(f"Errore: {e}")
# → Errore: Il divisore non può essere zero.
```

È come un cassiere che si rifiuta di accettare una banconota falsa: non aspetta che la banca se ne accorga — interviene subito.

Catturare un errore, fare qualcosa, e rilancerlo

A volte vuoi **registrare** che un errore è avvenuto (ad esempio scriverlo in un file di log), ma poi vuoi comunque che l'errore si propaghi al chiamante.

```
def leggi_file(percorso):
    try:
        with open(percorso) as f:
            return f.read()
    except FileNotFoundError as e:
        print(f"Log: il file '{percorso}' non è stato trovato.")
        raise # Rilancio lo stesso errore – non lo "ingoio"
```

Con `raise` da solo (senza argomenti), Python rilancia esattamente lo stesso errore che hai appena catturato.

Collegare gli errori tra loro (catena di eccezioni)

Puoi indicare che un errore è stato **causato** da un altro errore. Questo aiuta chi legge il messaggio di errore a capire l'intera catena di eventi.

```
def carica_configurazione(filename):
    try:
        with open(filename) as f:
            import json
            return json.load(f)
    except FileNotFoundError as e:
        # Lancio un errore più descrittivo, ma indico che è causato da "e"
        raise RuntimeError(f"Impossibile caricare la configurazione da '{filename}'") from e

try:
    config = carica_configurazione("config.json")
except RuntimeError as e:
    print(f"Errore: {e}")
    print(f"Causato da: {e.__cause__}")
```

La gerarchia degli errori

Gli errori in Python sono organizzati come un albero genealogico. Catturare un errore "genitore" significa catturare anche tutti i suoi "figli".

```
BaseException
├── Exception
│   ├── ArithmeticError
│   │   ├── ZeroDivisionError ← divisione per zero
│   │   └── OverflowError ← numero troppo grande
│   ├── LookupError
│   │   ├── IndexError ← indice lista fuori range
│   │   └── KeyError ← chiave dizionario non trovata
│   ├── ValueError ← valore sbagliato
│   ├── TypeError ← tipo sbagliato
│   └── OSError
│       └── FileNotFoundError ← file non trovato
```

```
try:
    x = [1, 2, 3][10]    # IndexError
except LookupError:
    # Cattura sia IndexError che KeyError, perché entrambi sono "figli" di
    # LookupError
    print("Errore di ricerca (indice o chiave non trovata)")
```

Context manager personalizzato (uso avanzato di `with`)

Hai già usato `with open(...)` per aprire file. Puoi creare i tuoi "context manager" che gestiscono automaticamente l'apertura e la chiusura di risorse — anche in caso di errore.

```
class GestoreRisorsa:
    def __enter__(self):
        # Questo viene eseguito quando si entra nel blocco "with"
        print("Risorsa aperta")
        return self

    def __exit__(self, tipo_errore, valore_errore, traceback):
        # Questo viene eseguito quando si esce dal blocco "with" (anche in
        # caso di errore)
        print("Risorsa chiusa")
        if tipo_errore:
            print(f"Si è verificato un errore: {valore_errore}")
        return False    # False = non "ingoia" l'errore, lascialo propagare

# Utilizzo
with GestoreRisorsa() as r:
    print("Sto usando la risorsa")
    # Se qui avviene un errore, __exit__ viene comunque chiamato
```

Buone pratiche

```
# MALE: cattura tutti gli errori senza distinzione
# Nasconde bug e rende difficile il debug
try:
    operazione_complessa()
except:
    pass # Silenzio totale – non sai cosa è andato storto

# BENE: cattura solo gli errori che ti aspetti
try:
    operazione_complessa()
except (ValueError, TypeError) as e:
    print(f"Errore gestito: {e}")

# BENE: registra l'errore e poi rilancia
try:
    operazione_complessa()
except Exception as e:
    logging.error(f"Errore inatteso: {e}")
    raise # Non nascondere l'errore – rilanciarlo
```

La regola d'oro: **cattura solo gli errori che sai come gestire**. Per tutti gli altri, lascia che si propaghino e vengano visti.